

TTTC2343

LAB06 TIAGO SIMULATION

Reference

<http://wiki.ros.org/Robots/TIAGo/Tutorials>

eBOOK ROS Robot Programming (Python and C++)

Instructions

Run all the TIAGo tutorials from Task 1-7 from the given link. Make a report to show the results of your simulation.

1)Tutorials Installation

1. [Installing Ubuntu with ROS + TIAGO](#)

This tutorial describes the steps needed to get a proper Ubuntu and ROS installation to have a system up and running for the TIAGo tutorials.

2. [Installing Tiago Tutorial Docker](#)

How to pull and launch a docker all set up for the Tiago Tutorial with melodic

3. [Testing the simulation](#)

Testing the simulation of Tiago in Gazebo

2) Control

1. [Teleoperating the mobile base with the keyboard](#)

How to move the differential drive base of TIAGo using the [key_teleop](#) package.

2. [Moving the base through velocity commands](#)

This tutorial shows how to move the base by sending velocity commands to the appropriate topic.

3. [Joint Trajectory Controller](#)

Example of how to move TIAGo's arm using a joint trajectory action.

4. [Moving individual joints](#)

Different ways to move individual joints of TIAGo are explained.

5. [Head control](#)

Example on how to move TIAGo's head using an action that makes the robot look to a given direction.

6. [Playing pre-defined upper body motions](#)

Tutorial on how to play back pre-defined upper body motions with TIAGo using the [play_motion](#) package.

3) Autonomous navigation

1. [Create a map with gmapping](#)

This tutorial shows how to create a map of the environment using the range-finder on the base of TIAGo.

2. [Localization and path planning](#)

Learn how to run laser-based localization and autonomous navigation avoiding obstacles by means of global and local path planning.

4) MoveIt!

1. [Planning in joint space](#)

How to reach a given joint space configuration using motion planning based on [MoveIt!](#)

2. [Planning in cartesian space](#)

Use [MoveIt!](#) to plan a joint trajectory in order to reach a given pose in cartesian space

3. [Planning in cartesian space with TRAC-IK](#)

Use [TRAC-IK](#) in [MoveIt!](#) to plan a joint trajectory in order to reach a given pose in cartesian space

4. [Planning with Octomap demo](#)

Use [Octomap](#) in [MoveIt!](#) to compute the collision checking with the environment around the robot during the motion planning of poses given in cartesian space

5. [Pick & Place demo](#)

Tabletop pick & place demo using monocular model-based object reconstruction based on [ArUco](#) markers and the pick and place pipeline in [MoveIt!](#)

5) OpenCV

1. [Track Sequential \(C++\)](#)

A simple method to detect and track basic movements/shapes on a static camera against a static background

2. [Corner Detection \(C++\)](#)

There are two corner detector algorithms often used in the OpenCV library, the Shi-Tomasi and Harris functions. In this simple tutorial you will see how changing two parameters can affect the corner detection

3. [Find Keypoints \(C++/Python\)](#)

OpenCV has a multitude of Feature detectors, and in this tutorial you will be able to go through most of them, and seeing how image sharpening and contrast affects the detection of features

4. [Matching \(C++/Python\)](#)

Using feature detection in two images, this class will attempt to find matches between the keypoints detected and thereby see if the image contains a certain object.

5. [ArUco marker detection \(C++\)](#)

This tutorial shows how to detect fiducial markers using the [ArUco](#) library and to get its 3D pose.

6. [Person detection \(C++\)](#)

ROS node using the OpenCV person detector based on HOG Adaboost cascade

7. [Face detection \(C++\)](#)

Example of ROS node embedding OpenCV's face detector.

8. [Planar object detection and pose estimation \(C++\)](#)

Planar textured object detection based on feature matching between live video feed and a reference image of the object. Then, the pose of the object is determined by homography estimation and provided the size of the object.

6) Point Cloud

1. [Table segmentation \(C++\)](#)

Example of plane segmentation applied to the detection of a table and the objects on top of it.

2. [Cylinder detector \(C++\)](#)

Cylindrical object detection based on sample consensus segmentation.

3. [Region Based Segmentation \(C++\)](#)

Region based segmentation takes a point in the pointcloud and determines whether neighbouring points are part of the same region

7) Multi-TIAGo

1. [Execute the Multi-TIAGo Simulation](#)

This tutorial shows how to launch the Multi-TIAGo simulation in Gazebo and how to execute some basic operations.

2. [Multi-TIAGo Navigation](#)

This tutorial shows how to launch the Multi-TIAGo navigation simulation in Gazebo and how to run autonomous navigation avoiding obstacles by means of global and local path planning.

"HAPPY ROBOTING"

Ts Dr AHAR